Machine Learning in Production

Automating and Testing ML Pipelines

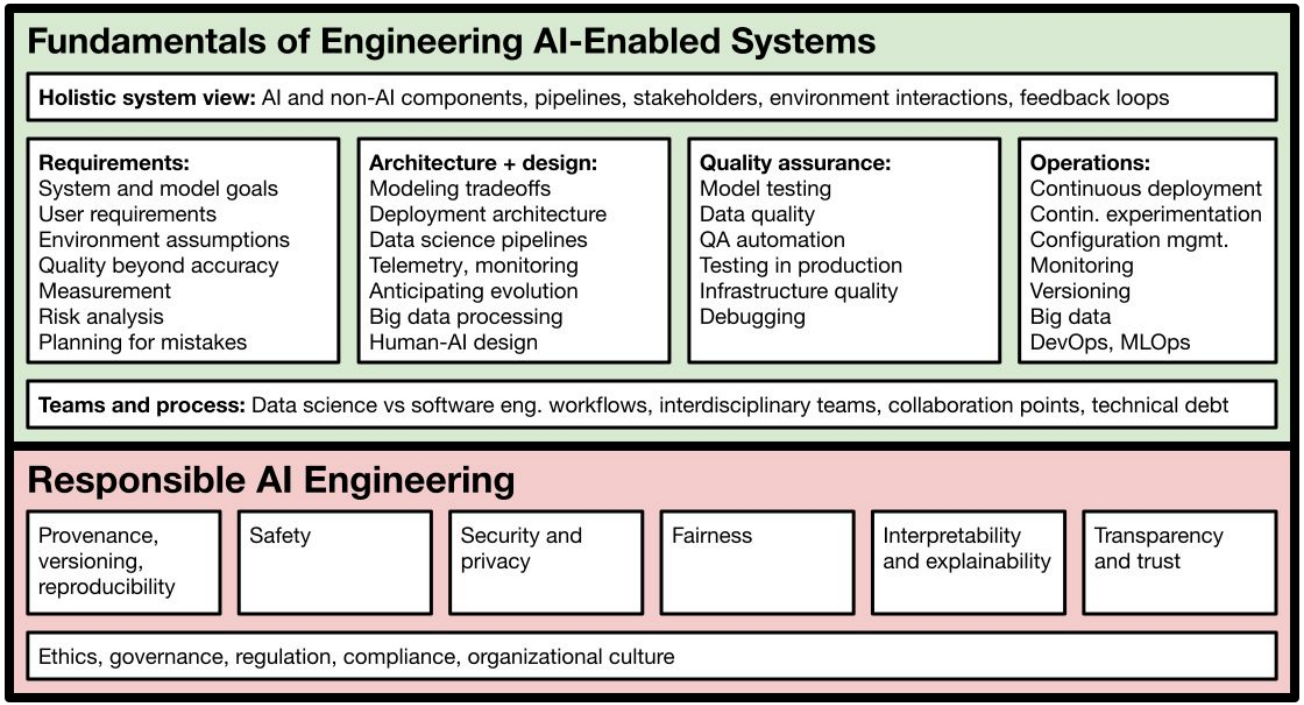
Administrativa

VM information sent out

VMs are already receiving recommendation requests

Team 18 was the first to respond

Infrastructure Quality...



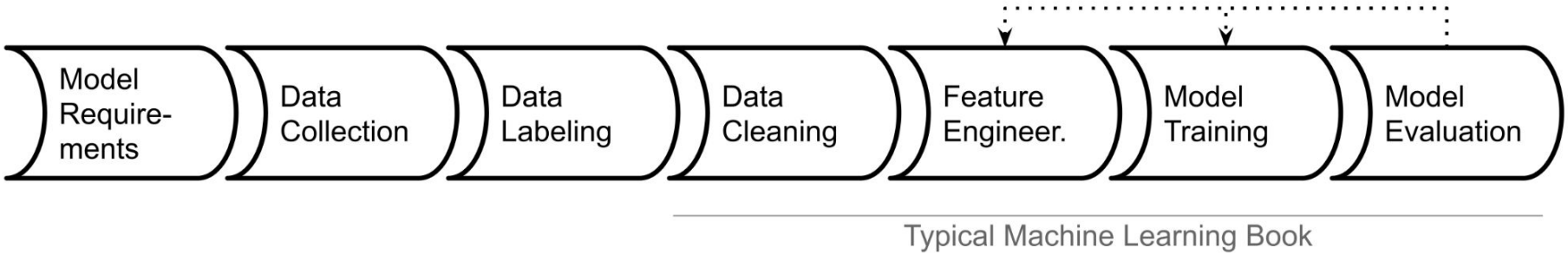
Readings

Required reading: Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, D. Sculley. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. Proceedings of IEEE Big Data (2017)

Learning Goals

- Decompose an ML pipeline into testable functions
- Implement and automate tests for all parts of the ML pipeline
- Understand testing opportunities beyond functional correctness
- Describe the different testing levels and testing opportunities at each level
- Automate test execution with continuous integration

Traditional Model Focus (Data Science)



Focus: building models from given data, evaluating accuracy

Common ML Pipeline

```
# load data collected from team1
import pandas as pd


url = 'http://128.2.25.78:8080/private/log1.clean'
df = pd.read_csv(url)
df.head()
```

	dayIdx	user	userAvgTime	location	dow	isWeekend	time
0	0	Pittsburgh66Correy	7.045001	Pittsburgh	6	True	0.000000
1	1	Pittsburgh66Correy	7.045001	Pittsburgh	7	True	6.883333
2	2	Pittsburgh66Correy	7.045001	Pittsburgh	1	False	6.816667
3	3	Pittsburgh66Correy	7.045001	Pittsburgh	2	False	7.383333
4	4	Pittsburgh66Correy	7.045001	Pittsburgh	3	False	0.000000

Data was preprocessed externally, identifying the time at a given day when the light was first turned on
12pm). Weather and sunrise information is not included here, though that'd be important. If the light w
this morning (quite common), 0 is recorded.

```
[ ] # just data encoding and splitting X and Y

    X = df.drop(['time'], axis=1)
    YnonZero = df['time'] > 0
    Y = df['time']

    from sklearn import preprocessing
    # leDate = preprocessing.LabelEncoder()
    # leDate.fit(X['date'])
    # leDate.transform(X['date'])

    X=X.apply(preprocessing.LabelEncoder().fit_transform)
    X
```

Notebooks as Production Pipeline?

Parameterize and use `nbconvert`?

What's Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities

Souti Chattopadhyay[1], **Ishita Prasad**[2], **Austin Z. Henley**[3], **Anita Sarma**[1], **Titus Barik**[2]
Oregon State University[1], Microsoft[2], University of Tennessee-Knoxville[3]
{chattops, anita.sarma}@oregonstate.edu, {ishita.prasad, titus.barik}@microsoft.com, azh@utk.edu

ABSTRACT
Computational notebooks—such as Azure, Databricks, and Jupyter—are a popular, interactive paradigm for data scientists to author code, analyze data, and interleave visualizations, all within a single document. Nevertheless, as data scientists incorporate more of their activities into notebooks, they encounter unexpected difficulties, or pain points, that impact their productivity and disrupt their workflow. Through a systematic, mixed-methods study using semi-structured interviews ($n = 20$) and survey ($n = 156$) with data scientists, we catalog nine pain points when working with notebooks. Our findings suggest that data scientists face numerous pain points throughout the entire workflow—from setting up notebooks to deploying to production—across many notebook environments. Our data scientists report essential notebook requirements, such as supporting data exploration and visualization. The results of our study inform and inspire the design of computational notebooks.

Author Keywords
Computational notebooks; challenges; data science; interviews; pain points; survey

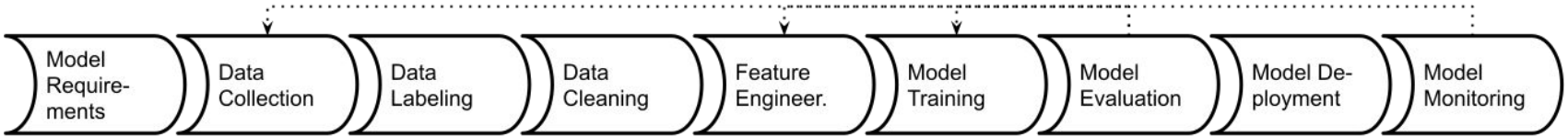
CCS Concepts
•**Human-centered computing** → **Interactive systems and**

Azure,[1] Databricks,[2] Colab,[3] Jupyter,[4] and nteract.[5] While originally intended for exploring and constructing computational narratives [29, 31], data scientists are now increasingly orchestrating more of their activities within this paradigm [33]: through long-running statistical models, transforming data at scale, collaborating with others, and executing notebooks directly in production pipelines. But as data scientists try to do so, they encounter unexpected difficulties—pain points—from limitations in affordances and features in the notebooks, which impact their productivity and disrupt their workflow.

To investigate the pain points and needs of data scientists who work in computational notebooks, across multiple notebook environments, we conducted a systematic mixed-method study using field observations, semi-structured interviews, and a confirmation survey with data science practitioners. While prior work has studied specific facets of difficulties in notebooks [24, 17], such as versioning [18, 19] or cleaning unused code [13, 34], the central contribution of this paper is a taxonomy of validated pain points across data scientists' notebook activities.

Our findings identify that data scientists face considerable pain points through the entire analytics workflow—from setting up the notebook to deploying to production—across

Possible Mistakes in ML Pipelines



Danger of "silent" mistakes in many phases

Examples?

Subtle Bugs in Data Wrangling Code

```
df['Join_year'] = df.Joined.dropna().map(
    lambda x: x.split(',')[1].split(' ')[1])
```

```
df.loc[idx_nan_age,'Age'].loc[idx_nan_age] =
    df['Title'].loc[idx_nan_age].map(map_means)
```

```
df["Weight"].astype(str).astype(int)
```

Subtle Bugs in Data Wrangling Code (continued)

```
df['Reviws'] = df['Reviews'].apply(int)
```

```
df["Release Clause"] =
    df["Release Clause"].replace(regex=['k'], value='000')
df["Release Clause"] =
    df["Release Clause"].astype(str).astype(float)
```

Subtle Bugs in Data Wrangling Code (continued)

```
num = data.Size.replace(r'[kM]+$', '', regex=True).
      astype(float)
factor = data.Size.str.extract(r'[\d\.]+([KM]+)',
                               expand =False)
factor = factor.replace(['k','M'], [10**3, 10**6]).fillna(1)
data['Size'] = num*factor.astype(int)
```

```
data["Size"]= data["Size"].
      replace(regex =['k'], value='000')
data["Size"]= data["Size"].
      replace(regex =['M'], value='000000')
data["Size"]= data["Size"].astype(str). astype(float)
```

Possible Mistakes in ML Pipelines

Danger of "silent" mistakes in many phases:

- Dropped data after format changes
- Failure to push updated model into production
- Incorrect feature extraction
- Use of stale dataset, wrong data source
- Data source no longer available (e.g web API)
- Telemetry server overloaded
- Negative feedback (telemtr.) no longer sent from app
- Use of old model learning code, stale hyperparameter
- Data format changes between ML pipeline steps

Pipeline Thinking

After exploration and prototyping build robust pipeline

One-off model creation -> repeatable automateable process

Enables updates, supports experimentation

Explicit interfaces with other parts of the system (data sources, labeling infrastructure, training infrastructure, deployment, ...)

Design for change

Building Robust Pipeline Automation

Support experimentation and evolution
- Automate
- Design for change
- Design for observability
- Testing the pipeline for robustness

Thinking in pipelines, not models

Integrating the Pipeline with other Components

Pipeline Testability and Modularity

Pipelines are Code

From experimental notebook code to production code

Each stage as a function or module

Well tested in isolation and together

Robust to changes in inputs (automatically adapt or crash, no silent mistakes)

Use good engineering practices (version control, documentation, testing, naming, code review)

Sequential Data Science Code in Notebooks

How to test??

```
# typical data science code from a notebook
df = pd.read_csv('data.csv', parse_dates=True)

# data cleaning
# ...

# feature engineering
df['month'] = pd.to_datetime(df['datetime']).dt.month
df['dayofweek']= pd.to_datetime(df['datetime']).dt.dayofweek
df['delivery_count'] = boxcox(df['delivery_count'], 0.4)
df.drop(['datetime'], axis=1, inplace=True)

dummies = pd.get_dummies(df, columns = ['month',  'weather', 'dayofweek'])
dummies = dummies.drop(['month_1', 'hour_0', 'weather_1'], axis=1)

X = dummies.drop(['delivery_count'], axis=1)
y = pd.Series(df['delivery_count'])

X_train, X_test, y_train, y_test =
            train_test_split(X, y, test_size=0.3, random_state=1)

# training and evaluation
lr = LinearRegression()
lr.fit(X_train, y_train)

print(lr.score(X_train, y_train))
print(lr.score(X_test, y_test))
```

Testability can be decomposed into...

Controllability:

- Ability to influence system internal state or behavior by changing its inputs

Observability

- Degree to which you can determine that the behavior went as expected.

Testable Code

Think about testing when writing code

Unit testing encourages you to write testable code

Separate parts of the code to make them independently testable

Abstract functionality behind interface, make it replaceable

Bonus: Test-Driven Development is a design and development method in which you *always* write tests *before* writing code

Pipeline restructured into separate functions

```
def encode_day_of_week(df):
   if 'datetime' not in df.columns: raise ValueError("Column datetime missing")
   if df.datetime.dtype != 'object': raise ValueError("Invalid type for column datetime")
   df['dayofweek']= pd.to_datetime(df['datetime']).dt.day_name()
   df = pd.get_dummies(df, columns = ['dayofweek'])
   return df

# ...

def prepare_data(df):
   df = clean_data(df)
   df = encode_day_of_week(df)
   df = encode_month(df)
   df = encode_weather(df)
   df.drop(['datetime'], axis=1, inplace=True)
   return (df.drop(['delivery_count'], axis=1),
           encode_count(pd.Series(df['delivery_count'])))

def learn(X, y):
   lr = LinearRegression()
   lr.fit(X, y)
   return lr

def pipeline():
   train = pd.read_csv('train.csv', parse_dates=True)
   test = pd.read_csv('test.csv', parse_dates=True)
   X_train, y_train = prepare_data(train)
   X_test, y_test = prepare_data(test)
   model = learn(X_train, y_train)
   accuracy = eval(model, X_test, y_test)
   return model, accuracy
```

Orchestrating Functions

```
def pipeline():
    train = pd.read_csv('train.csv', parse_dates=True)
    test = pd.read_csv('test.csv', parse_dates=True)
    X_train, y_train = prepare_data(train)
    X_test, y_test = prepare_data(test)
    model = learn(X_train, y_train)
    accuracy = eval(model, X_test, y_test)
    return model, accuracy
```

Dataflow frameworks like Luigi, DVC, Airflow, d6tflow, and Ploomber support distribution, fault tolerance, monitoring, ...

Hosted versions like DataBricks and AWS SageMaker Pipelines

Test the Modules

```
def encode_day_of_week(df):
   if 'datetime' not in df.columns: raise ValueError("Column datetime missing")
   if df.datetime.dtype != 'object': raise ValueError("Invalid type for column datetime")
   df['dayofweek']= pd.to_datetime(df['datetime']).dt.day_name()
   df = pd.get_dummies(df, columns = ['dayofweek'])
   return df
```

```
def test_day_of_week_encoding():
  df = pd.DataFrame({'datetime': ['2020-01-01','2020-01-02','2020-01-08'],
                     'delivery_count': [1, 2, 3]})
  encoded = encode_day_of_week(df)
  assert "dayofweek_Wednesday" in encoded.columns
  assert (encoded["dayofweek_Wednesday"] == [1, 0, 1]).all()

# more tests...
```

How to select test cases?

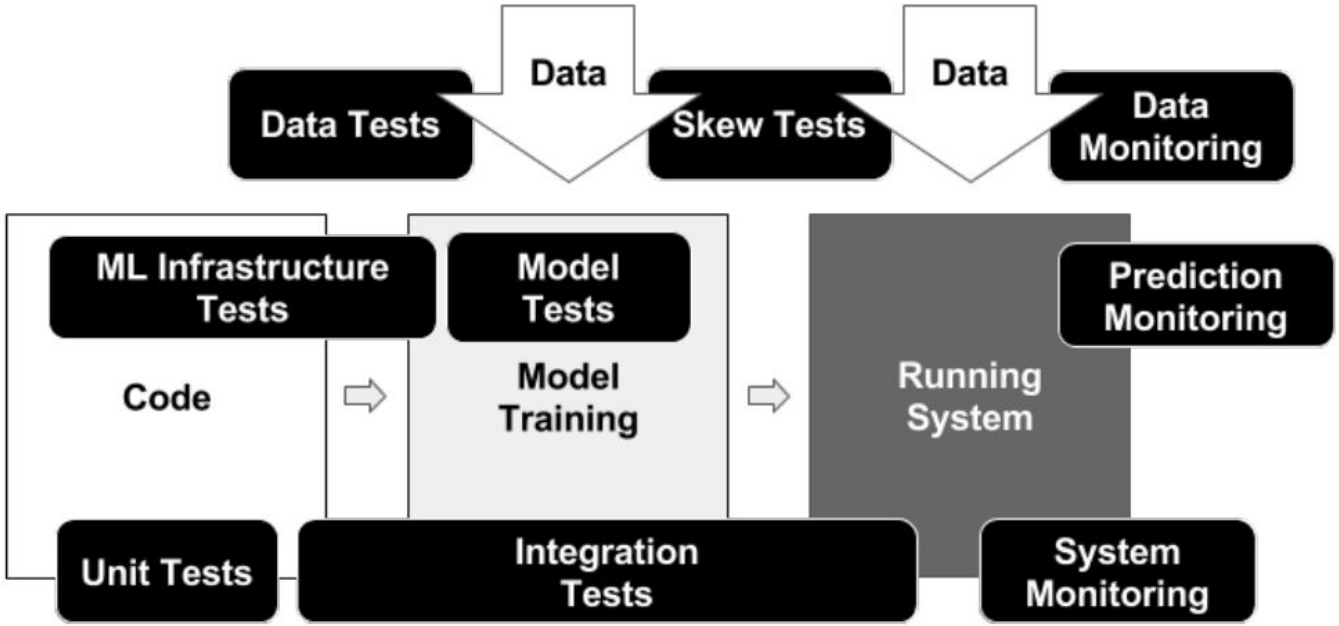
Modularity fosters Testability

Breaking code into functions/modules

Supports reuse, separate development, and testing

Can test individual parts

Testing Maturity


Data
Data
Data Tests
Skew Tests
Data Monitoring
ML Infrastructure Tests
Model Tests
Prediction Monitoring
Code
Model Training
Running System
Unit Tests
Integration Tests
System Monitoring

Data Tests

1. Feature expectations are captured in a schema.
2. All features are beneficial.
3. No feature's cost is too much.
4. Features adhere to meta-level requirements.
5. The data pipeline has appropriate privacy controls.
6. New features can be added quickly.
7. All input feature code is tested.

Tests for Model Development

1. Model specs are reviewed and submitted.
2. Offline and online metrics correlate.
3. All hyperparameters have been tuned.
4. The impact of model staleness is known.
5. A simpler model is not better.
6. Model quality is sufficient on important data slices.
7. The model is tested for considerations of inclusion.

ML Infrastructure Tests

1. Training is reproducible.
2. Model specs are unit tested.
3. The ML pipeline is Integration tested.
4. Model quality is validated before serving.
5. The model is debuggable.
6. Models are canaried before serving.
7. Serving models can be rolled back.

Monitoring Tests

1. Dependency changes result in notification.
2. Data invariants hold for inputs.
3. Training and serving are not skewed.
4. Models are not too stale.
5. Models are numerically stable.
6. Computing performance has not regressed.
7. Prediction quality has not regressed.

Case Study: Covid-19 Detection

(from S20 midterm; assume model deployment in the cloud)

Breakout Groups

In the Smartphone Covid Detection scenario, discuss in groups and post to #lecture, tagging group members:

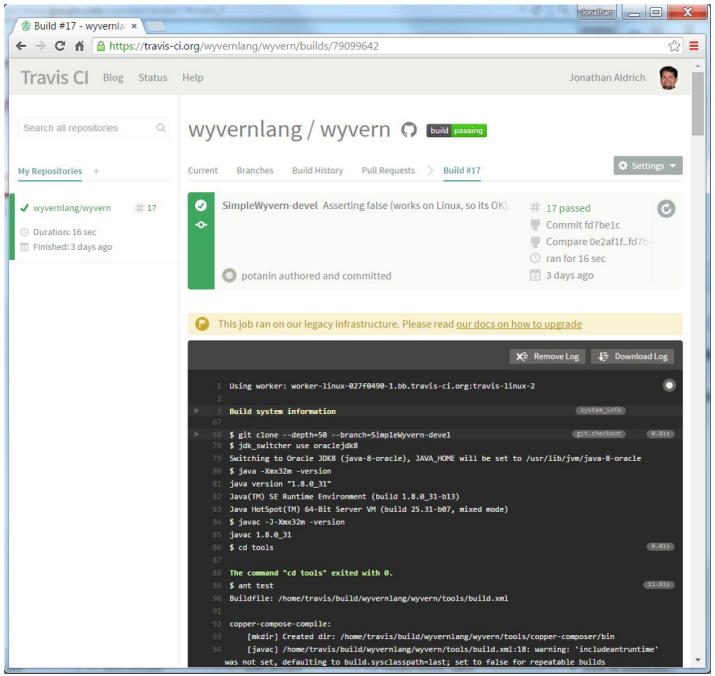
- Back (Claire's) left: data tests
- Back (Claire's) right: model dev. tests
- Front (Claire's) left: infrastructure tests
- Front (Claire's) right: monitoring tests

1. Considering all the test categories in the paper, discuss in the context of the Covid-detection scenario: **What would you do?**

2. Identify which discussed test you find most important and **sketch Python code** for it

Test Automation

From Manual Testing to Continuous Integration



Anatomy of a Unit Test

Specification

Controlled environment

Test inputs (calls and parameters)

Expected outputs/behavior (oracle)

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class AdjacencyListTest {
    @Test
    public void testSanityTest(){
        // set up
        Graph g1 = new AdjacencyListGraph(10);
        Vertex s1 = new Vertex("A");
        Vertex s2 = new Vertex("B");
        // check expected results (oracle)
        assertEquals(true, g1.addVertex(s1));
        assertEquals(true, g1.addVertex(s2));
        assertEquals(true, g1.addEdge(s1, s2));
        assertEquals(s2, g1.getNeighbors(s1)[0]);
    }

    // use abstraction, e.g. common setups
    private int helperMethod…
}
```

Unit Testing Pitfalls

Working code, failing tests

"Works on my machine"

Tests break frequently

How to avoid?

Build systems & Continuous Integration

Automate all build, analysis, test, and deployment steps from a command line call

Ensure all dependencies and configurations are defined

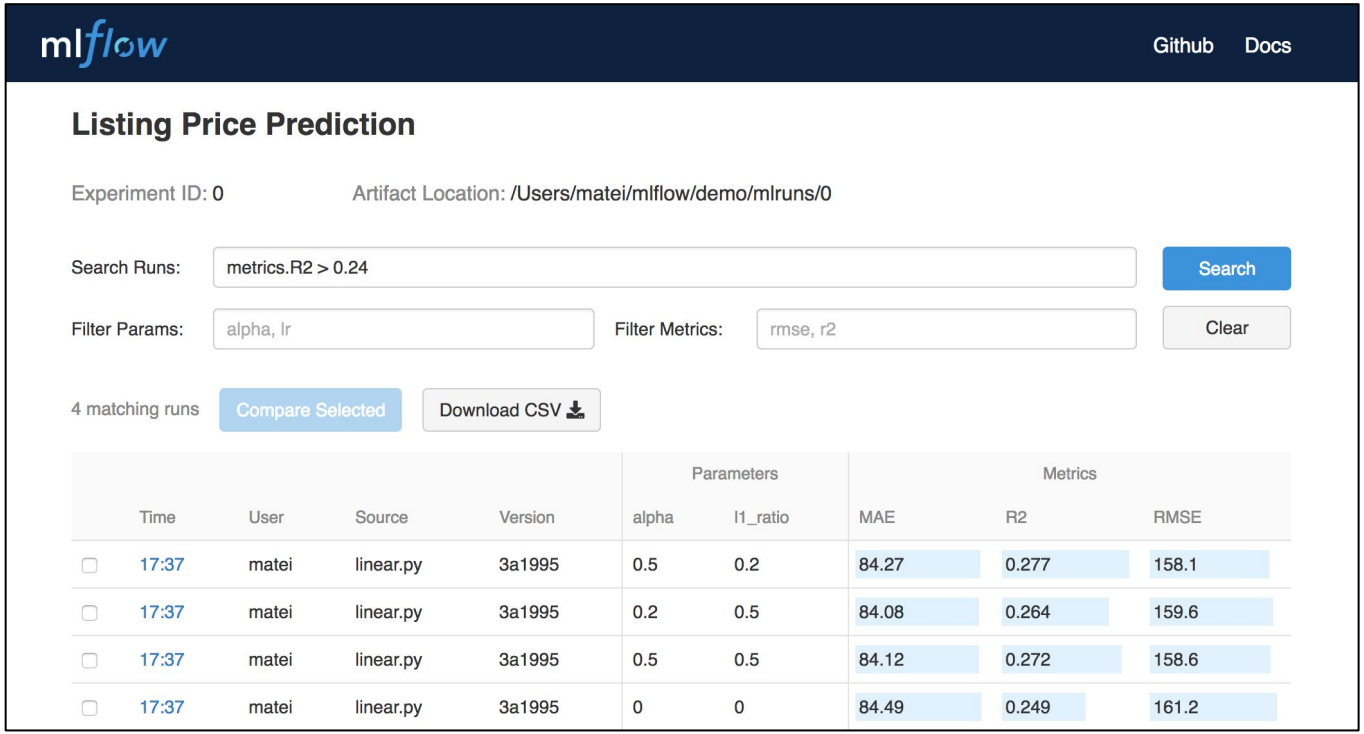
Ideally reproducible and incremental

Distribute work for large jobs

Track results

Key CI benefit: Tests are regularly executed, part of process

Tracking Model Qualities

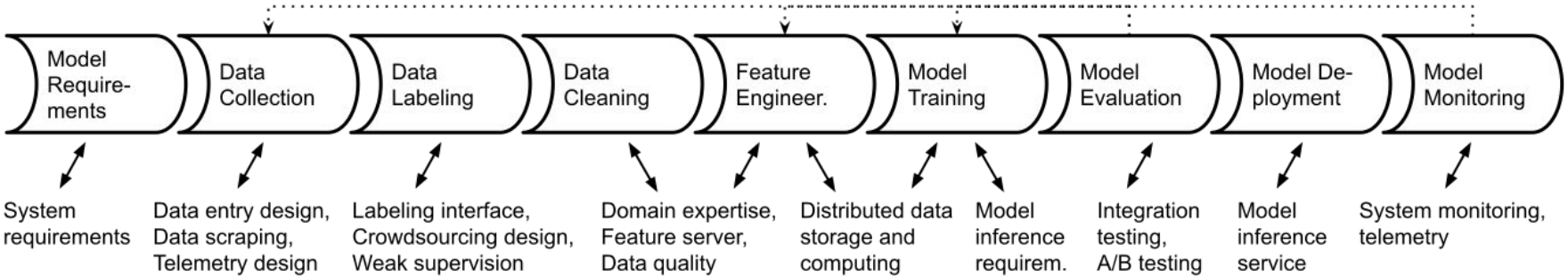


Many tools: MLFlow, Neptune, TensorBoard, Weights & Biases, Comet.ml, ...

MLFlow Example

```
1 import mlflow
2
3 mlflow.set_experiment("My first MLflow experiment")
4
5 with mlflow.start_run(run_name="First Run"):
6     mlflow.log_param("regularization", 0.5)
7     model1 = # ... model training code goes here
8     mlflow.log_metric("accuracy",
9                 accuracy(model1, validationData))
```

Real Pipelines can be Complex



Real Pipelines can be Complex

Large arguments of data

Distributed data storage

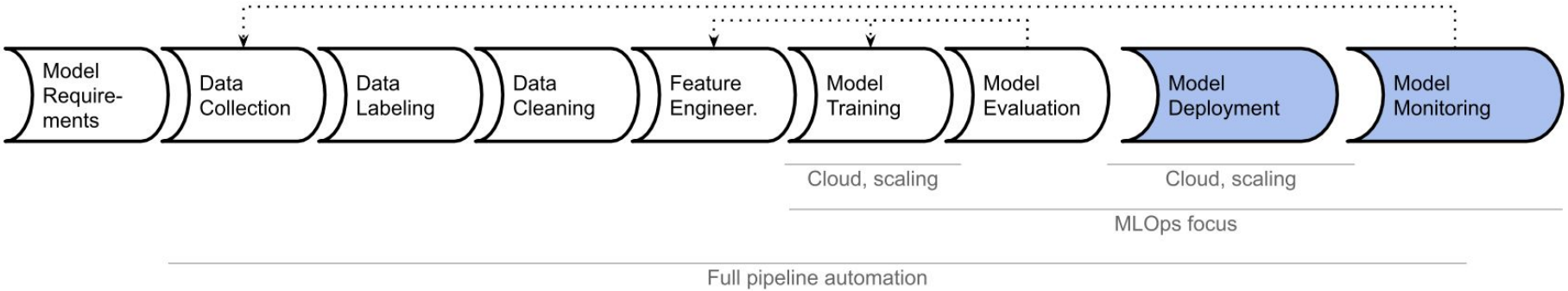
Distributed processing and learning

Special hardware needs

Fault tolerance

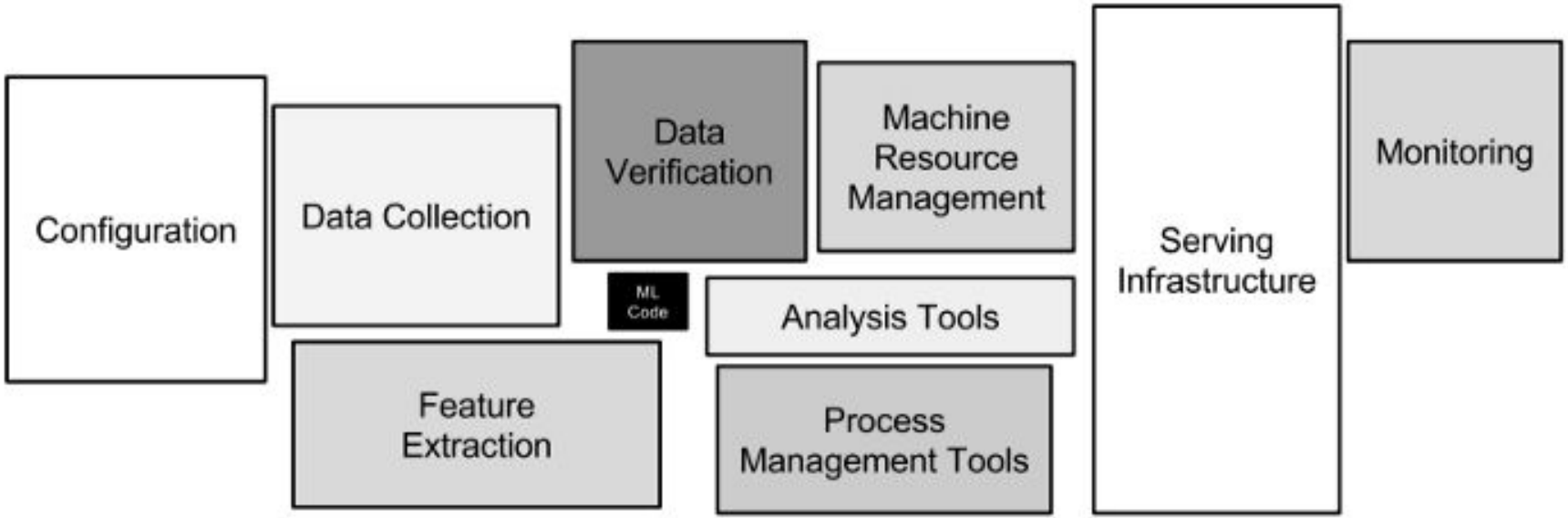
Humans in the loop

Automating Pipelines and MLOps (ML Engineering)



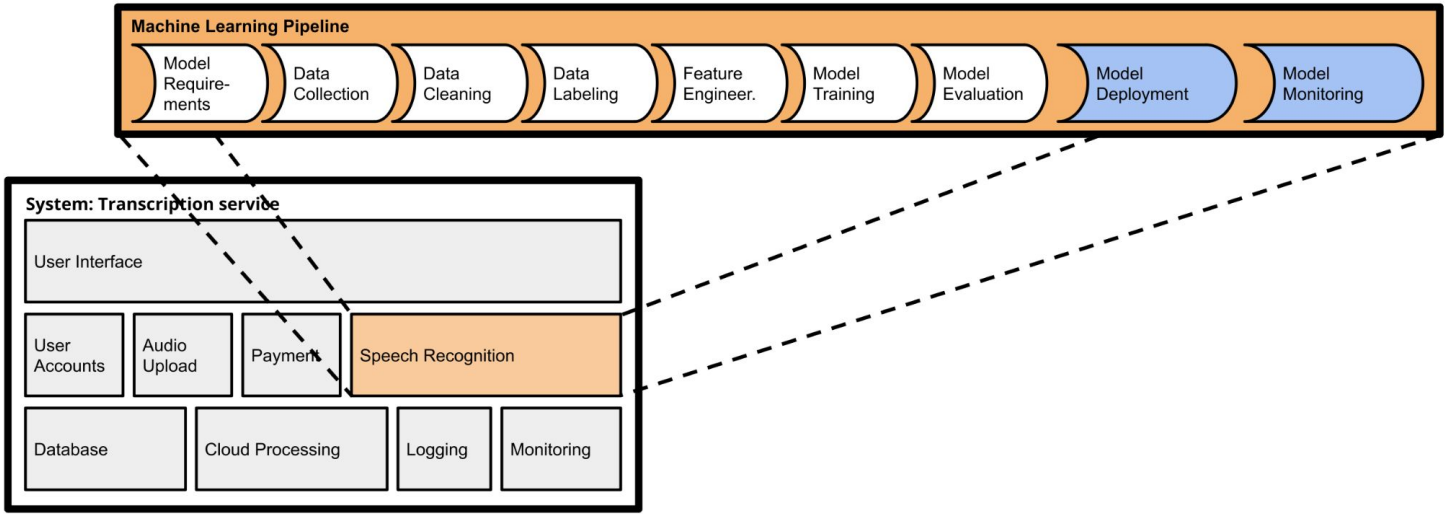
Focus: experimenting, deploying, scaling training and serving, model monitoring and updating

MLOps Infrastructure



From: Sculley, David, et al. “Hidden technical debt in machine learning systems.” NIPS 28 (2017).

ML-Powered Software



Interaction of ML and non-ML components, system requirements, user interactions, safety, collaboration, delivering products

Compound AI Systems

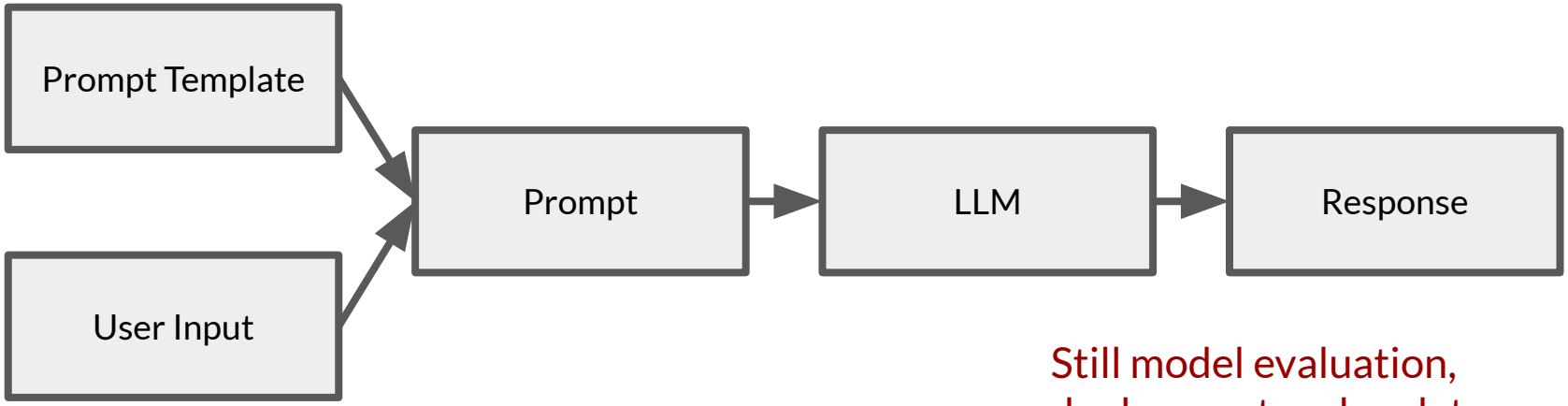
“Modern AI Pipelines”

Less focus on data wrangling, model training/fine tuning, more on prompting (“context engineering”)

Multiple LLM calls

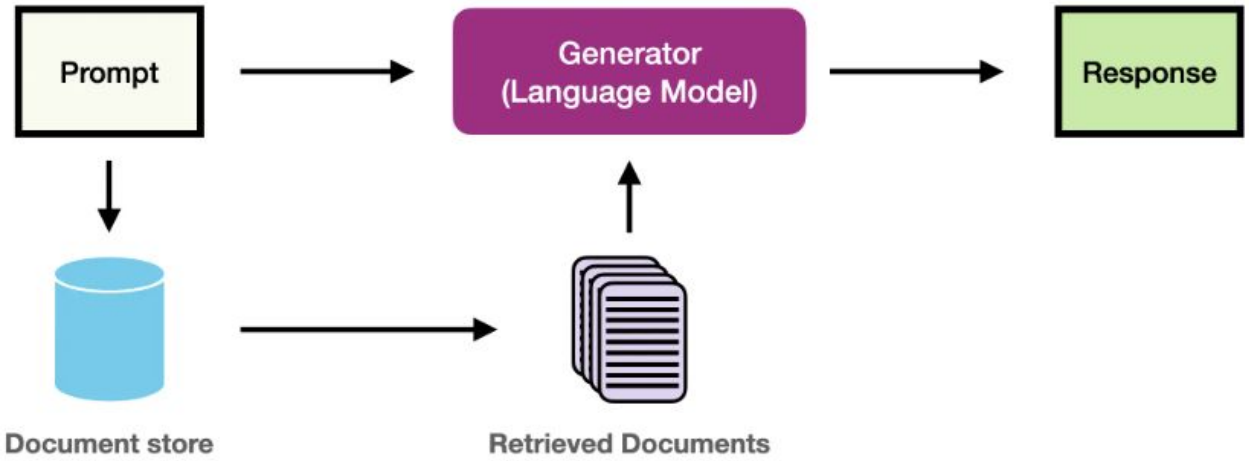
Integration with tools (retrieval, send message, …)

Deploying Prompt Templates



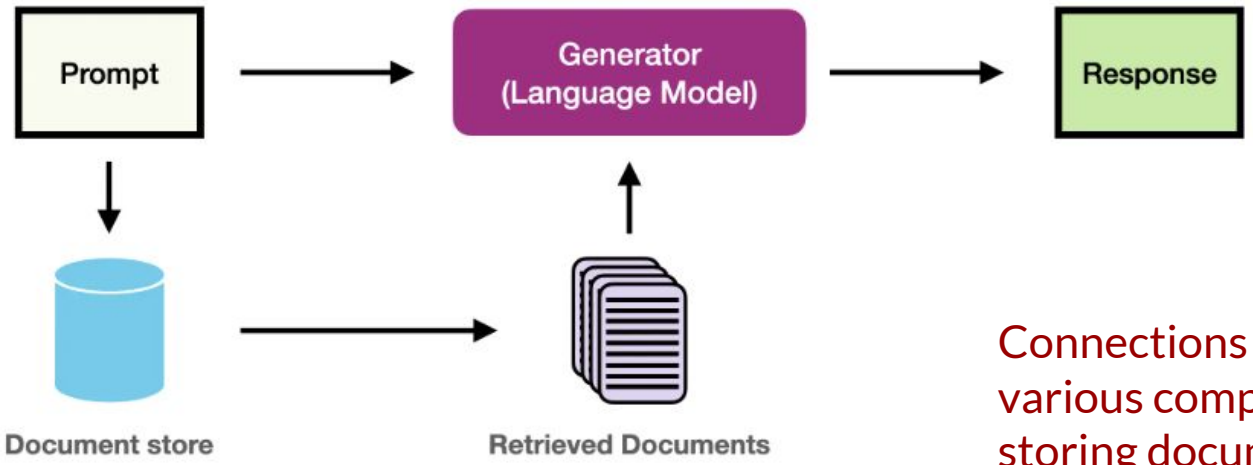
Still model evaluation, deployment and updates (to prompts and models), model monitoring, screening of user inputs, …

Retrieval Augmented Generation (RAG)



https://www.promptingguide.ai/research/rag

Retrieval Augmented Generation (RAG)

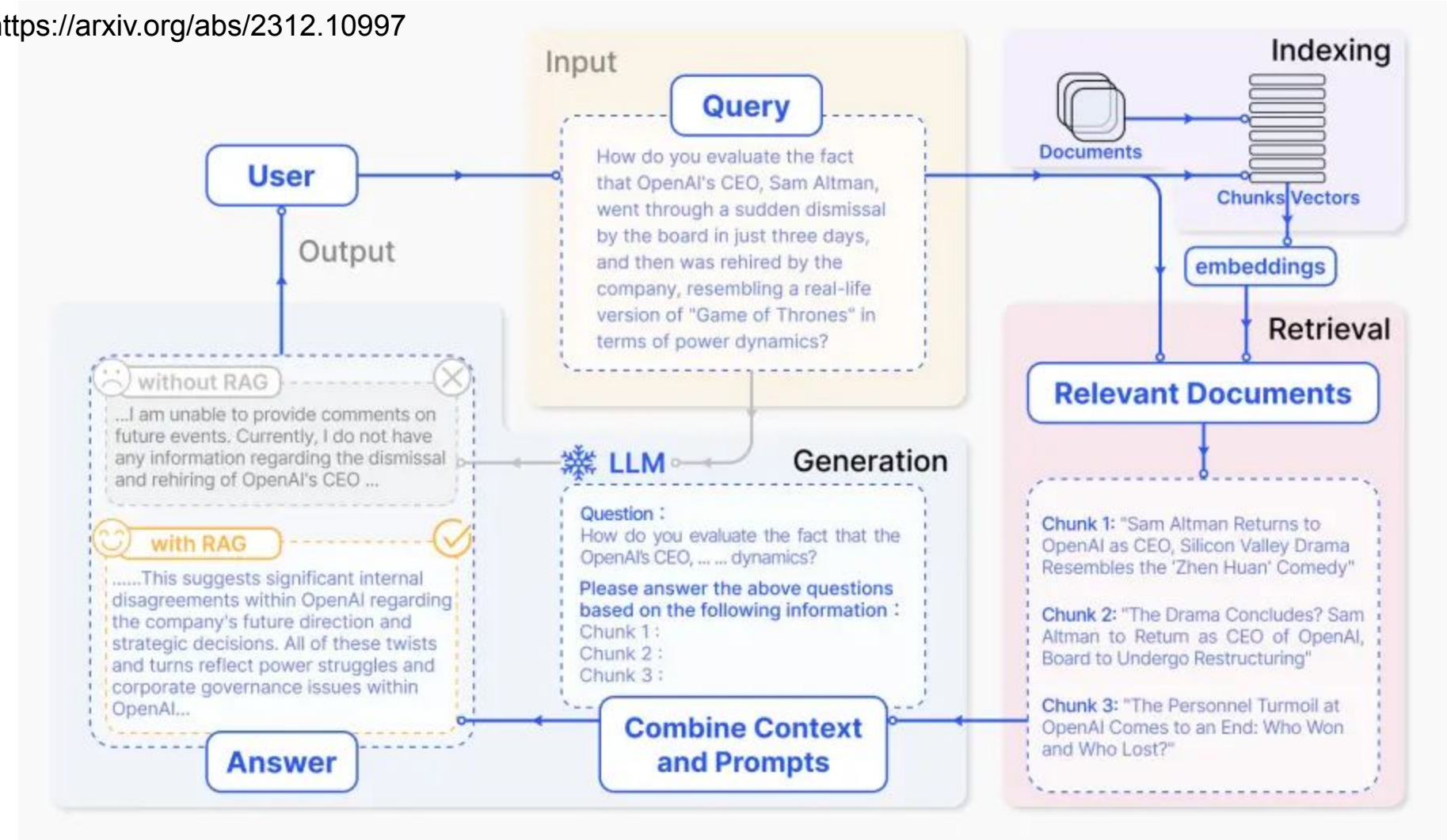


https://www.promptingguide.ai/research/rag

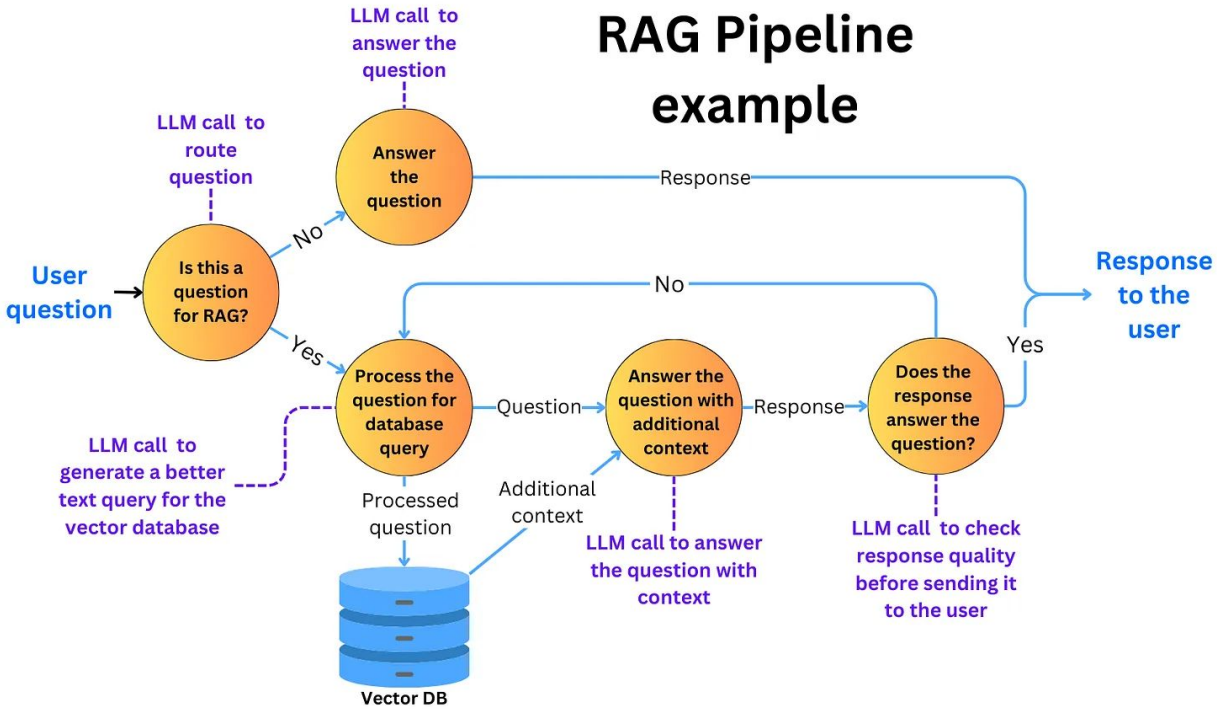
Connections to various components storing documents

Still model evaluation, model monitoring

https://arxiv.org/abs/2312.10997



Multi-Step Pipelines

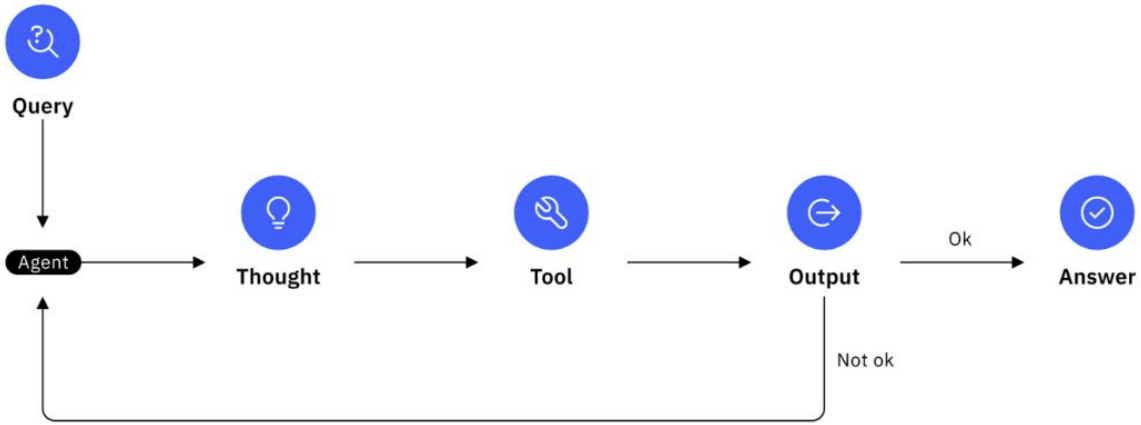


https://newsletter.theaiedge.io/p/how-to-optimize-llm-pipelines-with

AI Agents

```
def react_agent(query):
        context = llm(f"Think: How to answer '{query}'?")
        while not is_final_answer(context):
                action = llm(f"Context: {context}\nAction:")
                if "get_email" in action:
                        result = get_email(extract_date_range(action))
                elif "send_email" in action:
                        result = send_email(extract_to_body(action))
                context = llm(f"Previous: {context}\n
                        Action: {action}\nResult: {result}\nContext:")
        return llm(f"Final answer based on: {context}")
```

Inversion of control(!)

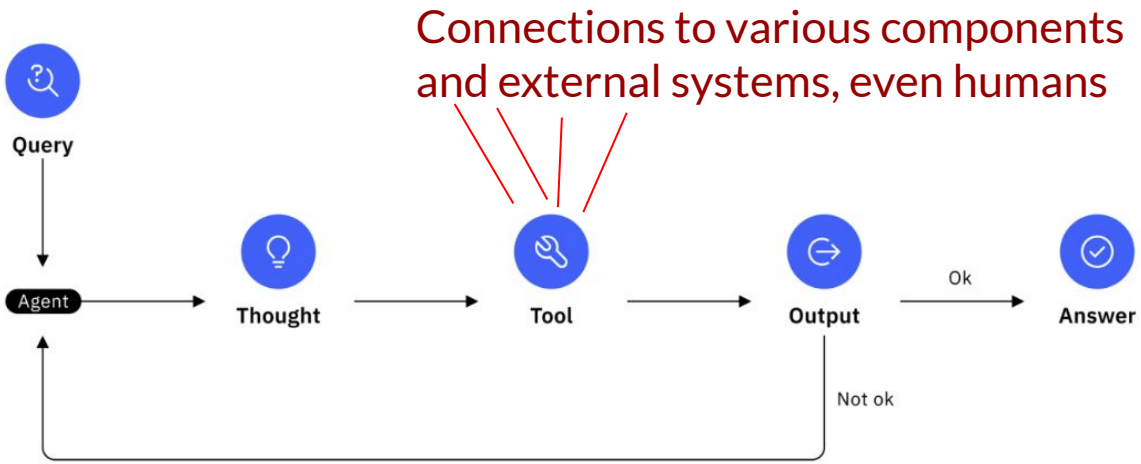


https://www.ibm.com/think/topics/react-agent

AI Agents

```
def react_agent(query):
        context = llm(f"Think: How to answer '{query}'?")
        while not is_final_answer(context):
                action = llm(f"Context: {context}\nAction:")
                if "get_email" in action:
                        result = get_email(extract_date_range(action))
                elif "send_email" in action:
                        result = send_email(extract_to_body(action))
                context = llm(f"Previous: {context}\n
                        Action: {action}\nResult: {result}\nContext:")
        return llm(f"Final answer based on: {context}")
```

Inversion of control(!)



https://www.ibm.com/think/topics/react-agent

Building Robust Compound AI Systems

Support experimentation and evolution
- Automate
- Design for change
- Design for observability
- Testing the pipeline for robustness

Thinking in workflows, not models

Integrating the Workflow with other Components

Testing across boundaries: Minimizing and Stubbing Dependencies

How to unit test component with dependency on other code?

How to Test Parts of a System?


```
# original implementation hardcodes external API
def clean_gender(df):
  def clean(row):
    if pd.isnull(row['gender']):
      row['gender'] = gender_api_client.predict(row['firstname'], row['lastname'], row['location'])
    return row
  return df.apply(clean, axis=1)
```

Automating Test Execution


```
def test_do_not_overwrite_gender():
   df = pd.DataFrame({'firstname': ['John', 'Jane', 'Jim'],
                      'lastname': ['Doe', 'Doe', 'Doe'],
                      'location': ['Pittsburgh, PA', 'Rome, Italy', 'Paris, PA '],
                      'gender': [np.nan, 'F', np.nan]})
   out = clean_gender(df, model_stub)
   assert(out['gender'] ==['M', 'F', 'M']).all()
```

Decoupling from Dependencies

```
def clean_gender_col(df, model):
  def clean(row):
    if pd.isnull(row['gender']):
      row['gender'] = model(row['firstname'],
                            row['lastname'],
                            row['location'])
    return row
  return df.apply(clean, axis=1)
```

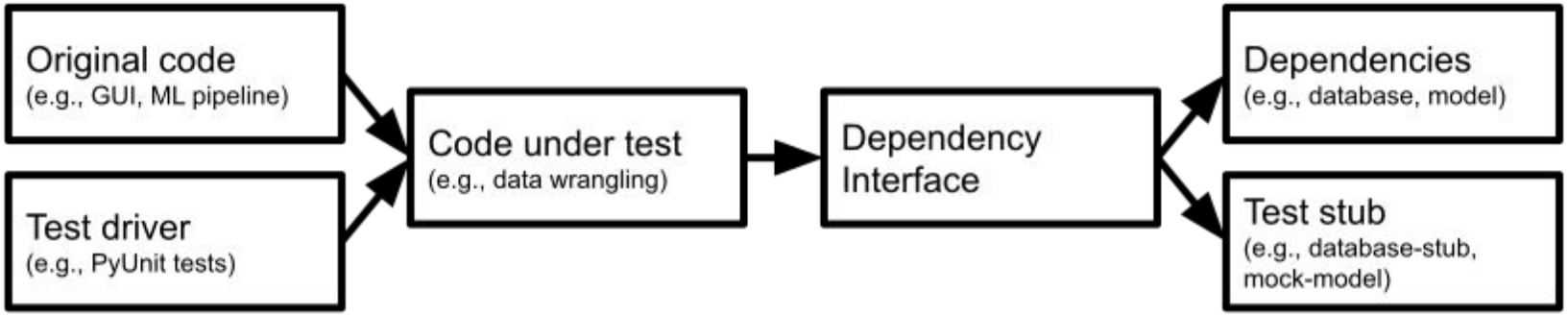
Replace concrete API with an interface that caller can parameterize

Stubbing the Dependency


```
def test_do_not_overwrite_gender():
  def model_stub(first, last, location):
    return 'M'

  df = pd.DataFrame({'firstname': ['John', 'Jane', 'Jim'],
                     'lastname': ['Doe', 'Doe', 'Doe'],
                     'location': ['Pittsburgh, PA', 'Rome, Italy', 'Paris, PA '],
                     'gender': [np.nan, 'F', np.nan]})
  out = clean_gender(df, model_stub)
  assert(out['gender'] ==['M', 'F', 'M']).all()
```

General Testing Strategy: Decoupling Code Under Test



(Mocking frameworks provide infrastructure for expressing such tests compactly.)

Testing Error Handling / Infrastructure Robustness

General Error Handling Strategies

Avoid silent errors

Recover locally if possible, propagate error if necessary -- fail entire task if needed

Explicitly handle exceptional conditions and mistakes

Test correct error handling

If logging only, is anybody analyzing log files?

Test for Expected Exceptions

```
def test_learning_fails_with_missing_data():
  df = pd.DataFrame({})
  with pytest.raises(NoDataError):
    learn(df)
```

Test Recovery Mechanisms with Stub

Use stubs to inject artificial faults

```python
## testing retry mechanism
from retry.api import retry_call
import pytest

# stub of a network connection, sometimes failing
class FailedConnection(Connection):
  remaining_failures = 0
  def __init__(self, failures):
    self.remaining_failures = failures
  def get(self, url):
    print(self.remaining_failures)
    self.remaining_failures -= 1
    if self.remaining_failures >= 0:
      raise TimeoutError('fail')
    return "success"

# function to be tested, with recovery mechanism
def get_data(connection, value):
  def get(): return connection.get('https://replicate.npmjs.com/registry/'+value)
  return retry_call(get,
       exceptions = TimeoutError, tries=3, delay=0.1, backoff=2)

# 3 tests for no problem, recoverable problem, and not recoverable
def test_no_problem_case():
  connection = FailedConnection(0)
  assert get_data(connection, '') == 'success'

def test_successful_recovery():
  connection = FailedConnection(2)
  assert get_data(connection, '') == 'success'

def test_exception_if_unable_to_recover():
  connection = FailedConnection(10)
  with pytest.raises(TimeoutError):
    get_data(connection, '')
```

Test Error Handling throughout Pipeline

Is invalid data rejected / repaired?

Are missing data updates raising errors?

Are unavailable APIs triggering errors?

Are failing deployments reported?

Log Error Occurrence

Even when reported or mitigated, log the issue

Allows later analysis of frequency and patterns

Monitoring systems can raise alarms for anomalies

Example: Error Logging

```
from prometheus_client import Counter
connection_timeout_counter = Counter(
          'connection_retry_total',
          'Retry attempts on failed connections')

class RetryLogger():
  def warning(self, fmt, error, delay):
    connection_timeout_counter.inc()

retry_logger = RetryLogger()

def get_data(connection, value):
  def get(): return connection.get('https://replicate.npmjs.com/registry/'+value)
  return retry_call(get,
       exceptions = TimeoutError, tries=3, delay=0.1, backoff=2,
       logger = retry_logger)
```

Test Monitoring

Inject/simulate faulty behavior

Mock out notification service used by monitoring

Assert notification

```
class MyNotificationService extends NotificationService {
    public boolean receivedNotification = false;
    public void sendNotification(String msg) {
        receivedNotification = true; }
}
@Test void test() {
    Server s = getServer();
    MyNotificationService n = new MyNotificationService();
    Monitor m = new Monitor(s, n);
    s.stop();
    s.request(); s.request();
    wait();
    assert(n.receivedNotification);
}
```

Testing Compound AI System

```
def test_agent_terminates():
    """Agent should reach a final answer within step budget."""
    result = react_agent("What meetings do I have tomorrow?",
                         max_steps=10)
    assert result.terminated
    assert result.steps_taken <= 10


def test_agent_handles_tool_failure():
    """Agent should gracefully handle a tool that errors out."""
    def failing_calendar(date_range):
        raise ConnectionError("Calendar API unavailable")
    result = react_agent("What meetings do I have tomorrow?",
                         tools={"get_calendar": failing_calendar},
                         max_steps=10)
    assert "unavailable" in result.answer.lower() or result.terminated
```

Test Monitoring in Production

Like fire drills (manual tests may be okay!)

Manual tests in production, repeat regularly

Actually take down service or trigger wrong signal to monitor

Where to Focus Testing?

Testing in ML Pipelines

Usually assume ML libraries already tested (pandas, sklearn, etc)

Focus on custom code

- data quality checks
- data wrangling (feature engineering)
- training setup
- interaction with other components

Consider tests of latency, throughput, memory, ...

Testing Data Quality Checks

Test correct detection of problems

```
def test_invalid_day_of_week_data():
  ...
```

Test correct error handling or repair of detected problems

```
def test_fill_missing_gender():
  ...
def test_exception_for_missing_data():
  ...
```

Test Data Wrangling Code

```
num = data.Size.replace(r'[kM]+$', '', regex=True).
      astype(float)
factor = data.Size.str.extract(r'[\d\.]+([KM]+)',
                               expand =False)
factor = factor.replace(['k','M'], [10**3, 10**6]).fillna(1)
data['Size'] = num*factor.astype(int)
```

```
data["Size"]= data["Size"].
      replace(regex =['k'], value='000')
data["Size"]= data["Size"].
      replace(regex =['M'], value='000000')
data["Size"]= data["Size"].astype(str). astype(float)
```

Test Model Training Setup?

Execute training with small sample data

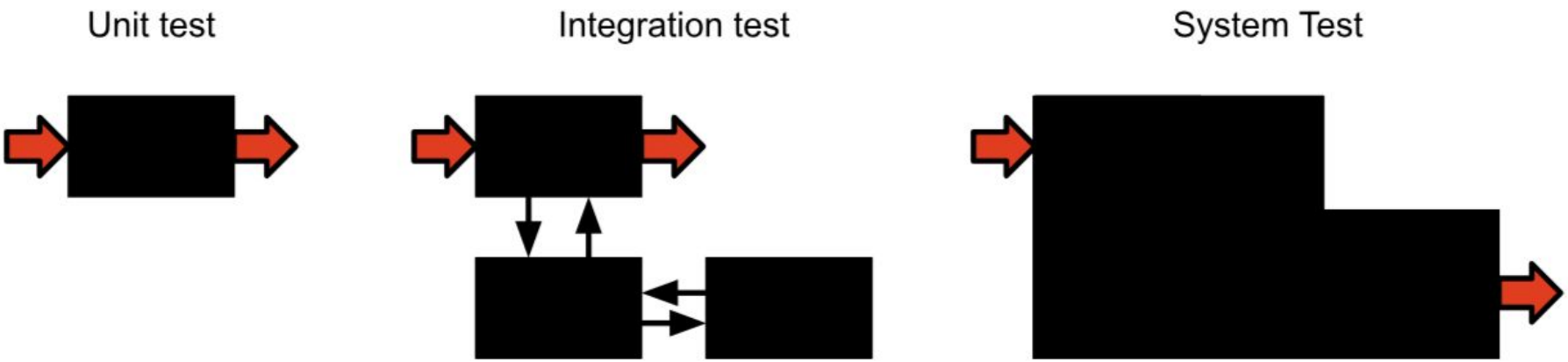
Ensure shape of model and data as expected (e.g., tensor dimensions)

Test Interactions with Other Components

Test error handling for detecting connection/data problems

- loading training data
- feature server
- uploading serialized model
- A/B testing infrastructure

Integration and system tests



Integration and system tests

Test larger units of behavior

Often based on use cases or user stories -- from the customer's perspective

```
@Test void gameTest() {
    Poker game = new Poker();
    Player p = new Player();
    Player q = new Player();
    game.shuffle(seed)
    game.add(p);
    game.add(q);
    game.deal();
    p.bet(100);
    q.bet(100);
    p.call();
    q.fold();
    assert(game.winner() == p);
}
```

Integration tests

```
def test_cleaning_with_feature_eng() {
    d = load_test_data();
    cd = clean(d);
    f = feature3.encode(cd);
    assert(no_missing_values(f["m"]));
    assert(max(f["m"]) <= 1.0);
}
```

Test combined behavior of multiple functions

Test Integration of Components

```
// making predictions with an ensemble of models
function predict_price(data, models, timeoutms) {
   // send asynchronous REST requests all models
   const requests = models.map(model => rpc(model, data, {timeout: timeoutms}).
                               then(parseResult).catch(e => -1))
   // collect all answers and return average if at least two models succeeded
   return Promise.all(requests).then(predictions => {
       const success = predictions.filter(v => v >= 0)
       if (success.length < 2) throw new Error("Too many models failed")
       return success.reduce((a, b) => a + b, 0) / success.length
   })
}

// test ensemble of models
const timeout = 500, M1 = "http://localhost:3000/predict", ...
beforeAll(() => {
  // launch model 1 API at address M1
  // launch model 2 API at address M2
  // launch model API with timeout at address M3
}
afterAll(() => { /* shut down all model APIs */ }

test("success despite timeout", async () => {
  const start = performance.now();
  const val = await predict_price(input, [M1, M2, M3], timeout)
  expect(performance.now() - start).toBeLessThan(2 * timeout)
  expect(val).toBeGreaterThan(0)
})

test("fail on too many timeouts", async () => {
  const start = performance.now();
  const val = await predict_price(input, [M1, M3, M3], timeout)
  expect(performance.now() - start).toBeLessThan(2 * timeout)
```

End-To-End Test of Entire Pipeline

```
def test_pipeline():
  train = pd.read_csv('pipelinetest_training.csv', parse_dates=True)
  test = pd.read_csv('pipelinetest_test.csv', parse_dates=True)
  X_train, y_train = prepare_data(train)
  X_test, y_test = prepare_data(test)
  model = learn(X_train, y_train)
  accuracy = eval(model, X_test, y_test)
  assert accuracy > 0.9
```

System Testing from a User Perspective

Test the product as a whole, not just components

Click through user interface, achieve task
(often manually performed)

Derived from requirements (use cases, user stories)

Testing in production

Bonus: Data Linter at Google

Miscoding

- Number, date, time as string
- Enum as real
- Tokenizable string (long strings, all unique)
- Zip code as number

Outliers and scaling

- Unnormalized feature (varies widely)
- Tailed distributions
- Uncommon sign

Packaging

- Duplicate rows
- Empty/missing data

Further readings: Hynes, Nick, D. Sculley, and Michael Terry. The data linter: Lightweight, automated sanity checking for ML data sets. NIPS MLSys Workshop. 2017.

Summary

Summary

- Beyond model and data quality: Quality of the infrastructure matters, danger of silent mistakes
- Automate pipelines to foster testing, evolution, and experimentation
- Many SE techniques for test automation, testing robustness, test adequacy, testing in production useful for infrastructure quality

Further Readings

- O'Leary, Katie, and Makoto Uchida. "Common problems with Creating Machine Learning Pipelines from Existing Code." Proc. Third Conference on Machine Learning and Systems (MLSys) (2020).
- Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, D. Sculley. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. Proceedings of IEEE Big Data (2017)
- Zinkevich, Martin. Rules of Machine Learning: Best Practices for ML Engineering. Google Blog Post, 2017
- Serban, Alex, Koen van der Blom, Holger Hoos, and Joost Visser. "Adoption and Effects of Software Engineering Best Practices in Machine Learning." In Proc. ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (2020).